

University Course Management System

Software Verification and Validation Project Report



Submitted By:

Jawaria tahreem – 095

Maryam Sajid – 107

Maryyum Amin - 180

Department of Software Engineering

Table of Contents

Introduction.....	3
Project Overview	3
Problem Statement.....	4
Objectives of the Project.....	4
Scope of the System.....	5
Requirement Engineering	5
Functional Requirements	6
Non Functional Requirements	6
Requirement Defect Analysis	7
GitHub Repository Management	7
Z Notation Modeling.....	8
VDM Specification	8
Alloy Structural Verification	9
UI Prototype Development	10
GitHub Actions CI Pipeline.....	10
Validation and Testing.....	11
OWASP ZAP Security Analysis.....	11
Results and Findings	12
Challenges Faced	13
Lessons Learned.....	13
Future Improvements	13
Conclusion	14
References.....	14

Introduction

Software Verification and Validation is one of the most important areas of software engineering because it ensures that software systems are reliable, secure, and correct according to user requirements. Verification focuses on checking whether the software is built correctly while validation ensures that the correct system is being developed. Both activities are necessary for improving software quality and reducing system failures.

For this semester project, our group developed a University Course Management System called UCMS. The project was completed by following a complete Software Verification and Validation pipeline. The project included requirement engineering, formal specification, frontend prototype development, structural verification, validation activities, security analysis, and repository management.

The purpose of this project was not only to create a software prototype but also to understand how software correctness and quality can be improved using formal methods and testing activities. The project also helped us understand practical software engineering concepts such as continuous integration, repository management, and security testing.

Different tools such as React, GitHub, GitHub Actions, Alloy Analyzer, VDM, Z notation, and OWASP ZAP were used successfully during project development. The project improved our practical understanding of verification and validation techniques.

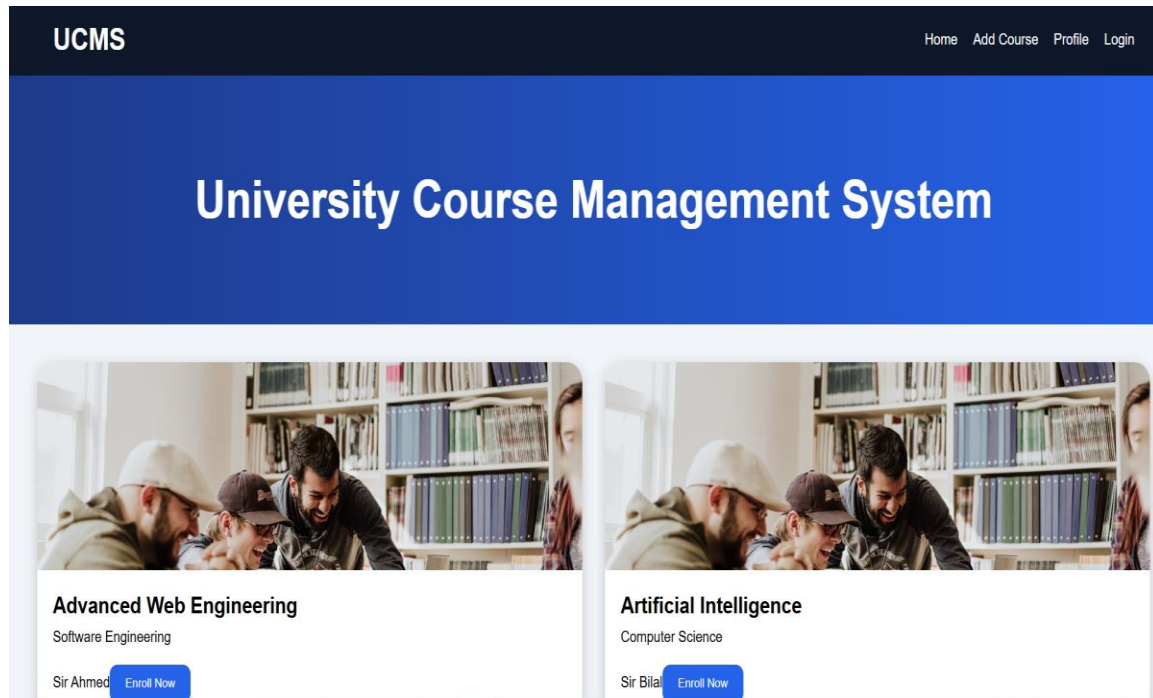
Project Overview

The University Course Management System is a web based academic management system designed to organize university related activities efficiently. The system provides an organized environment where students and teachers can interact with course information and academic records.

The system allows students to view available courses, access schedules, and manage course enrollments. Teachers can manage courses and update academic information. The frontend prototype was developed using React and Vite to create a responsive and user friendly interface.

The project mainly focused on verification and validation activities instead of creating a fully commercial system. Formal methods were used to define and verify system behavior mathematically. GitHub was used for repository management while GitHub Actions automated the validation process using a CI pipeline.

The project demonstrates how software quality can be improved through proper documentation, formal specification, testing, and validation activities.



Problem Statement

Educational institutions often face difficulties while managing academic records manually or through outdated systems. Manual systems can create many problems such as data inconsistency, scheduling conflicts, duplicate records, communication delays, and human errors.

Students may face issues while checking course availability or schedules. Teachers may also experience difficulty while updating academic information and managing enrolled students. Without proper verification activities, software systems may become unreliable and difficult to maintain.

The purpose of the UCMS project was to provide a simple and organized solution for managing university courses while applying verification and validation techniques. The project also aimed to improve software reliability, consistency, and maintainability.

Objectives of the Project

The main objectives of this project are:

- To develop a University Course Management System prototype.

- To apply Software Verification and Validation concepts practically.
- To create formal specifications using Alloy, VDM, and Z notation.
- To organize project files using GitHub.
- To integrate GitHub Actions Continuous Integration pipeline.
- To perform security testing using OWASP ZAP.
- To improve understanding of formal methods.
- To improve software quality and system consistency.
- To create professional project documentation.
- To understand practical frontend development using React.

The project was designed to combine theoretical verification techniques with practical software development activities.

Scope of the System

The scope of the UCMS project mainly focuses on academic course management activities. The system was developed as a frontend prototype with verification and validation support.

The project includes:

- Student registration
- Course viewing
- Course enrollment
- Schedule management
- UI prototype development
- Formal specification
- Structural verification
- Validation activities
- Repository management
- Security testing

The project does not include commercial level deployment, online payment systems, or real database hosting. The main focus remained on software verification and validation activities.

Requirement Engineering

Requirement Engineering was the first stage of the project. During this phase, all project requirements were identified, analyzed, and documented carefully.

The Software Requirement Specification document was prepared to define system

features and quality requirements. Requirements were divided into functional and non functional categories.

Requirement analysis helped improve understanding of system behavior and reduced ambiguity during development. Proper requirement engineering also improved communication among team members and ensured that all project objectives were clearly defined.

This phase played an important role in the success of later verification and validation activities because formal specifications depend heavily on correct requirement analysis.

Functional Requirements

Functional requirements describe the operations and features that the system should perform.

The main functional requirements of UCMS include:

- Students should be able to view available courses.
- Students should be able to enroll in courses.
- Students should be able to access schedules.
- Teachers should manage course information.
- Admin should manage academic records.
- The system should display course details properly.
- The interface should support navigation between pages.

These requirements helped define the expected behavior of the system clearly.

Non Functional Requirements

Non functional requirements describe quality related characteristics of the system.

The important non functional requirements include:

- The interface should remain user friendly.
- The UI should be responsive.
- The system should maintain consistency.
- Repository files should remain organized.
- The project should support validation activities.
- The system should remain easy to maintain.
- The application should support security testing.

Non functional requirements helped improve software quality and usability.

Requirement Defect Analysis

Requirement defect analysis was performed to identify issues such as ambiguity, inconsistency, and unclear statements.

Some requirements initially contained vague wording that could lead to misunderstanding during development. Requirement defects were documented carefully and GitHub Issues were used to track and manage them.

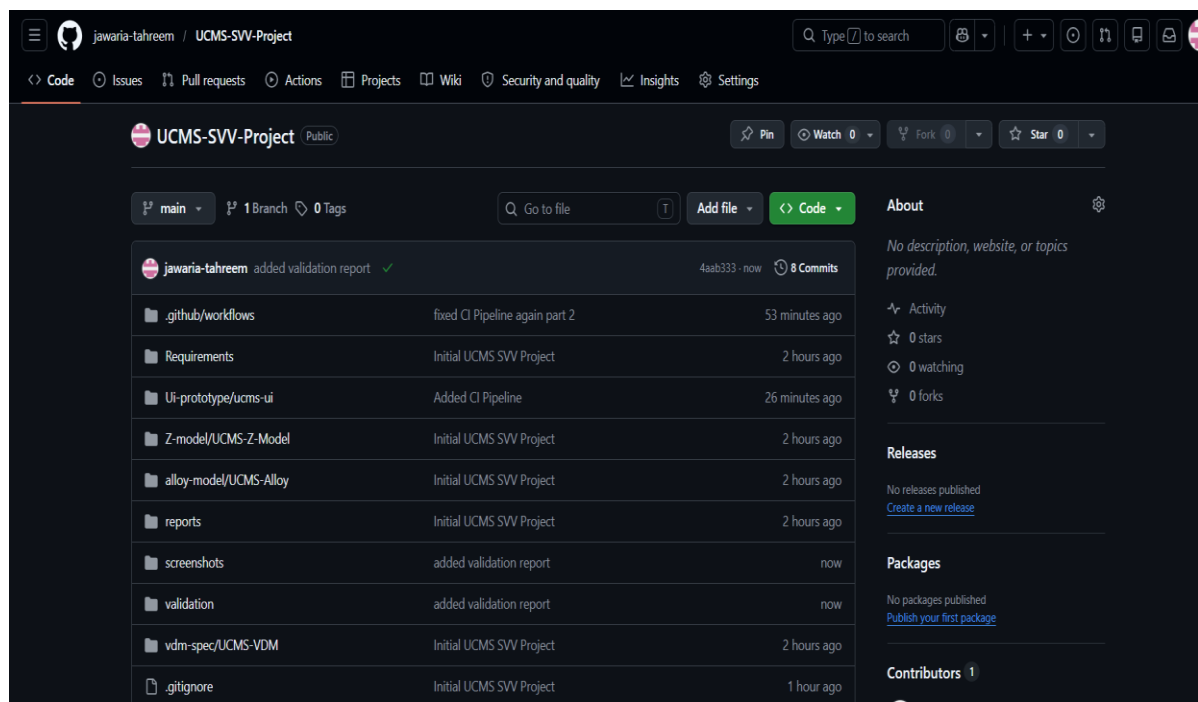
This activity improved understanding of how incorrect requirements can affect software quality. Requirement defect analysis also helped improve project documentation and requirement clarity.

GitHub Repository Management

GitHub was used for repository management and version control during the entire project.

Different folders were created to organize project files properly. The repository included folders for reports, screenshots, validation files, Alloy models, VDM specifications, Z notation files, requirements, and UI prototype.

The repository structure followed the project guidelines provided for the semester project. GitHub Actions workflows were also added inside the .github/workflows directory.



GitHub helped maintain project organization and simplified collaboration among team members.

Z Notation Modeling

Z notation was used to formally define system states and operations mathematically. The Z model improved precision and reduced ambiguity in system behavior.

Different schemas were created to represent initialization, enrollment, and course dropping operations. Invariant conditions were also defined to maintain system consistency.

The Z specification included:

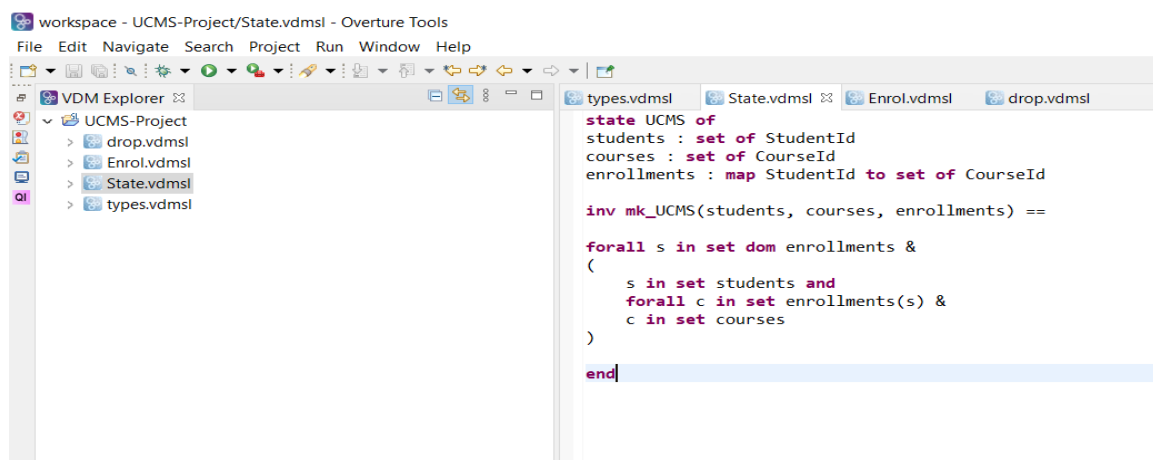
- State schemas
- Initialization schema
- Enrollment operation
- Drop operation
- Invariant rules

Using Z notation improved understanding of formal verification techniques and mathematical modeling.

VDM Specification

VDM was used to define operations, state transitions, and validation rules formally. The specification included data types, preconditions, and postconditions.

The VDM model represented academic activities such as student enrollment and course management. Formal specification helped improve understanding of system functionality and contract based verification.



The VDM specification also helped ensure that operations satisfy expected conditions before and after execution. Rest of the screenshots are added in the compressed folder that is submitted online.

Alloy Structural Verification

Alloy Analyzer was used for lightweight formal modeling and structural verification.

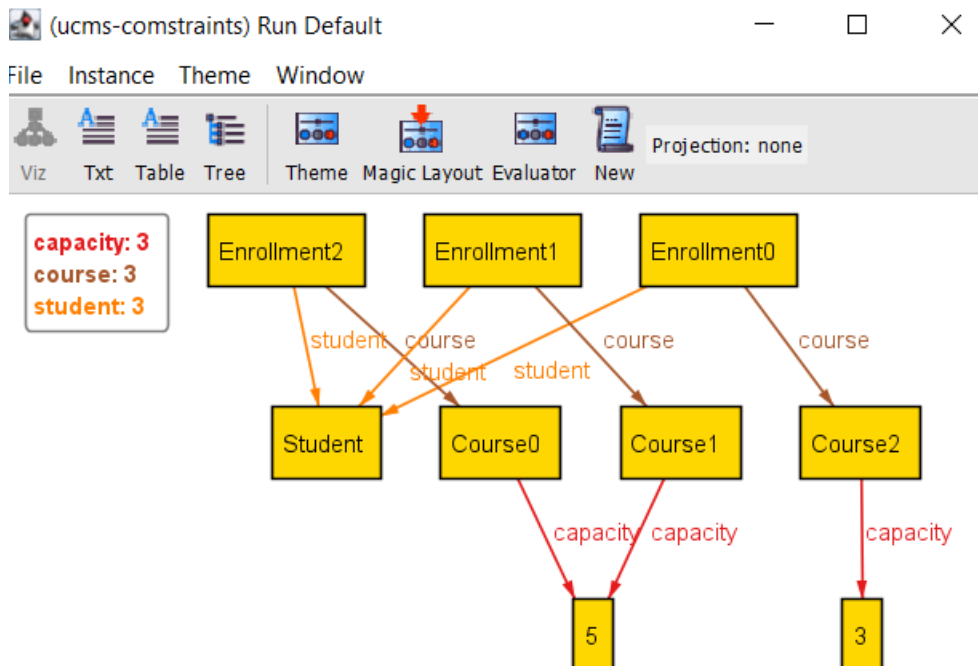
Different entities such as students, courses, and enrollments were modeled using signatures and relations. Facts and assertions were added to maintain consistency and verify system correctness.

The Alloy Analyzer generated visual instances that helped understand system relationships more clearly. Counterexample analysis was also performed to identify possible inconsistencies.

The Alloy model included:

- Signatures
- Relations
- Facts
- Assertions
- Predicates
- Counterexample analysis

This activity improved understanding of structural verification and logical analysis. Rest of the alloy analysis is shown in the compressed file which is submitted in the classroom.



UI Prototype Development

The frontend prototype of the UCMS project was developed using React and Vite. Different pages and components were designed to simulate a university management environment.

The UI included:

- Homepage
- Student dashboard
- Navigation bar
- Login form
- Registration page
- Course management page

CSS styling was applied to improve the appearance of the system. The interface was designed to remain simple, responsive, and easy to use.

Component based development in React helped improve code organization and reusability. The UI prototype successfully demonstrated how the final system could appear in a real implementation.

GitHub Actions CI Pipeline

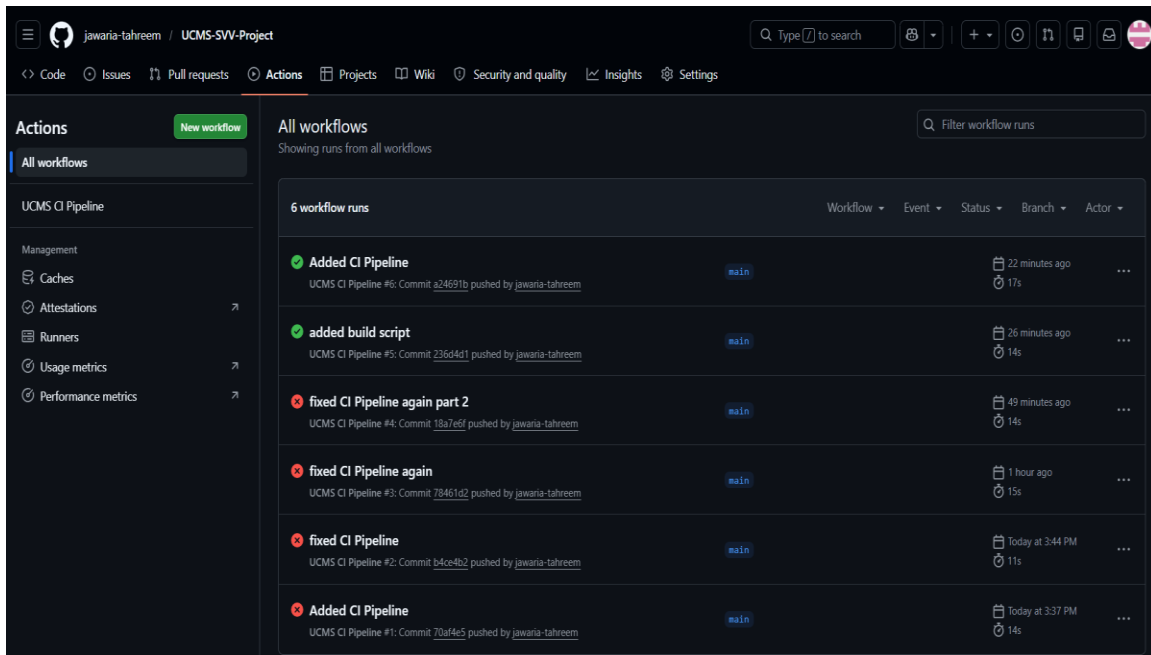
A Continuous Integration pipeline was integrated using GitHub Actions. The workflow automatically validated the React project whenever updates were pushed to the repository.

The workflow performed:

- Repository checkout
- Node.js setup
- Dependency installation
- React build validation

Several issues initially occurred due to incorrect folder paths and missing build scripts. These problems were solved successfully through debugging and workflow configuration updates.

The CI pipeline improved project automation and helped validate builds automatically.



Validation and Testing

Validation and testing activities were performed throughout the project to ensure software quality and correctness.

Different testing methods included:

- Functional testing
- Repository validation
- Build validation
- Formal verification
- UI testing

The GitHub Actions pipeline validated the React application automatically. Formal verification activities using Alloy, Z notation, and VDM improved understanding of software consistency and correctness.

Validation activities ensured that the project satisfied the defined requirements successfully.

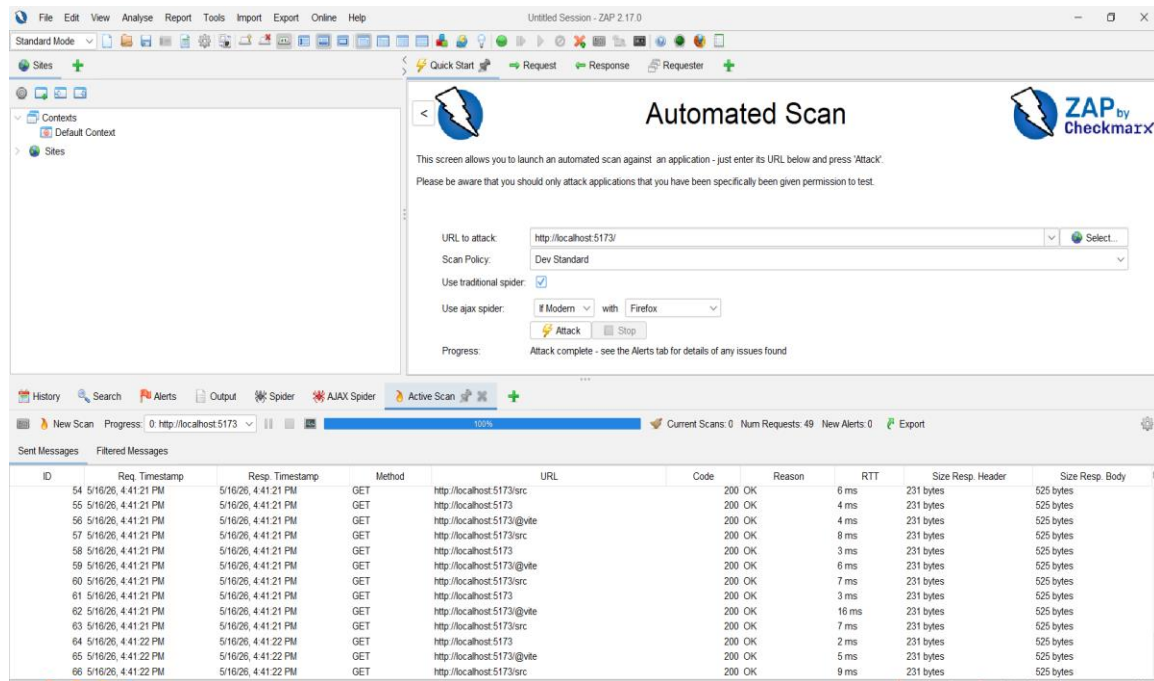
OWASP ZAP Security Analysis

OWASP ZAP was used for automated security testing of the React application.

The application was scanned locally using localhost during development. The scan identified warnings related to configuration weaknesses and missing security headers.

Security testing improved understanding of software vulnerabilities and secure software development practices. Reports and screenshots generated by OWASP ZAP were stored inside the validation folder.

The activity also demonstrated the importance of security analysis during software development.



Results and Findings

The project successfully demonstrated practical implementation of Software Verification and Validation concepts.

The React frontend prototype worked correctly after dependency configuration and workflow fixes. The GitHub Actions CI pipeline successfully validated the build process automatically.

Formal specifications using Alloy, Z notation, and VDM improved understanding of system correctness and logical consistency. Security testing using OWASP ZAP also provided valuable insights regarding software vulnerabilities.

Overall, the project objectives were achieved successfully and the system prototype performed according to expectations.

Challenges Faced

Several challenges were faced during project development.

One major issue involved GitHub Actions workflow configuration. Folder path mismatches and missing build scripts caused workflow failures initially. These issues were solved after debugging and configuration updates.

Another challenge involved setting up OWASP ZAP correctly because the React application server needed to remain active during scanning.

Managing multiple project files and maintaining repository organization also required proper teamwork and communication.

Despite these challenges, all major issues were solved successfully.

Lessons Learned

This project provided valuable learning experience regarding software verification and validation activities.

The project improved our understanding of:

- Formal methods
- Continuous Integration
- GitHub repository management
- Security testing
- Frontend development
- Software documentation
- Team collaboration

The project also improved practical debugging and problem solving skills.

Future Improvements

The current UCMS project is a prototype and can be improved further in future versions.

Possible improvements include:

- Database integration

- Authentication system
- Cloud deployment
- API integration
- Advanced dashboards
- Notification systems
- Better security mechanisms
- Real time updates

Future development can transform the prototype into a complete real world academic management system.

Conclusion

The University Course Management System project successfully demonstrated implementation of Software Verification and Validation concepts in a practical environment.

The project included requirement engineering, formal specification, frontend prototype development, CI pipeline integration, validation activities, and security testing.

Different tools such as React, GitHub Actions, Alloy Analyzer, VDM, Z notation, and OWASP ZAP were used successfully throughout the project.

The project improved understanding of software quality assurance, formal verification, testing, and repository management. Overall, the project achieved its objectives successfully and provided valuable learning experience.

References

React Documentation

GitHub Documentation

GitHub Actions Documentation

OWASP ZAP Documentation

Alloy Analyzer Documentation

VDM Documentation

Software Verification and Validation Course Material